CISC instructions.

Way Point 6

1. *We have successfully designed a hardwired processor that implements the entire SimpleRisc ISA.*
2. *Our processor is broadly divided into five stages: IF, OF, EX, MA, and RW.*
3. *The data path contains state elements (such as registers), arithmetic units, logical units, and multiplexers to choose the right set of inputs for each functional unit.*
4. *The multiplexers are controlled by control signals generated by the control unit.*

9.4 Microprogram-Based Processor

Let us now look at a different paradigm for designing processors. We have up till now looked at a processor with a hardwired control unit. We designed a data path with all the elements required to process, and execute an instruction. Where there was a choice between the input operands, we added a multiplexer that was controlled by a signal from the control unit. The control unit took the contents of the instruction as the input, and generated all the control signals. This design style is typically adopted by modern high performance processors. Note that efficiency comes at a cost. The cost is *flexibility*. It is fairly difficult for us to introduce new instructions. We need to possibly add more multiplexers, and generate many more control signals for each new instruction. Secondly, it is not possible to add new instructions to a processor after it has been shipped to the customer. Sometimes, we desire such flexibility.

It is possible to introduce this additional flexibility by introducing a translation table that translates instructions in the ISA to a set of simple microinstructions. Each microinstruction has access to all the latches, and internal state elements of a processor. By executing a group of microinstructions associated with an instruction, we can realize the functionality of that instruction. These microinstructions or *microcodes* are saved in a microcode table. It is typically possible to modify the contents of this table via software, and thus change the way hardware executes instructions. There are several reasons for wanting such kind of flexibility that allows us to add new instructions, or modify the behavior of existing instructions. Some reasons are as follows:

Definition 60

We can have an alternate design style, where we break instructions in the ISA to a set of microinstructions (microcodes). For each instruction, a dedicated unit executes its associated set of microinstructions to implement its functionality. It is typically possible to dynamically change the set of microinstructions associated with an instruction. This helps

us change the functionality of the instruction via software. Such a processor is known as a microprogrammed *processor.*

1. Processors sometimes have bugs in the execution of certain instructions [Sarangi et al., 2006]. This is because of mistakes committed by designers in the design process, or due to manufacturing defects. One such famous example is the bug in division in the Intel® Pentium® processor. Intel had to recall all the Pentium processors that it had sold to customers [Pratt, 1995]. If it had been possible to dynamically change the implementation of the division instruction, then it would not have been necessary to recall all the processors. Hence, we can conclude that some degree of reconfigurability of the processor can help us fix defects that might have been introduced in various stages of the design and manufacturing process.

2. Processors such as Intel Pentium 4, and later processors such as Intel® Core™i3, and Intel® Core™i7 implement some complex instructions by executing a set of microinstructions saved in memory. Complicated operations with strings of data, or instructions that lead to a series of repetitive computations are typically implemented using microcode. This means that the Intel processor internally replaces a complex instruction with a snippet of code containing simpler instructions. This makes it easier for the processor to implement complicated instructions. We do not need to unnecessarily make changes to the data path, add extra state, multiplexers, and control signals to implement complex instructions.

3. Nowadays, processors are part of a chip with many other elements. This is known as a system-on-chip (SOC). For example, a chip in a cellphone might contain a processor, a video controller, an interface to the camera, a sound and network controller. Processor vendors typically hardwire a set of simple instructions, and a lot of other instructions for interfacing with peripherals such as the video and audio controllers are written in microcode. Depending on the application domain and the set of peripheral components, the microcode can be customized.

4. Sometimes custom diagnostic routines are written using a set of dedicated microinstructions. These routines test different parts of the chip during its operation, report faults, and take corrective action. These built-in-self-test (BIST) routines are typically customizable, and are written in microcode. For example, if we desire high reliability, then we can modify the behavior of instructions that perform reliability checks on the CPU to check all components. However, in the interest of time, these routines can be compressed to check fewer components.

We thus observe that there are some compelling reasons to be able to programatically alter the behavior of instructions in a processor to achieve reliability, implement additional features, and improve portability. Hence, modern computing systems, especially, smaller devices such as phones, and tablets use chips that rely on microcode. Such microcode sequences are popularly referred to as *firmware*.

Definition 61
Modern computing systems, especially, smaller devices such as phones, modems, printers, and tablets use chips that rely on microcode. Such microcode sequences are popularly referred to as firmware*.*

Let us thus design a microprogram-based processor that provides us significantly more flexibility in tailoring the instruction set, even after the processor has been fabricated and sent to the customer. Before we proceed to design the data path and control path of a microprogrammed processor, we need to note that there is a fundamental tradeoff between a regular hardwired processor as presented in Section 9.2.7, and a microprogrammed processor. The tradeoff is efficiency versus flexibility. We cannot expect to have a very flexible processor that is fast and power efficient. Let us keep this important tenet in mind and proceed with the design.

9.5 Microprogrammed Data Path

Let us design the data path for a microprogrammed processor. Let us not design it from scratch. Let us rather modify the data path for the processor as shown in Section 9.2.7. Recall that it had some major units such as the fetch unit, register file, ALU, branch unit, and memory unit. These units were connected with wires, and whenever there was a possibility of multiple source operands, we added a multiplexer in the data path. The role of the control unit was to generate all the control signals for the multiplexers.

The issue is that the connections to the multiplexers are hardwired. It is not possible to establish arbitrary connections. For example, it is not possible to send the output of the memory unit to the input of the execute unit. Hence, we wish to have a design that is free of fixed interconnections between components. It should be theoretically possible for any unit to send data to any other unit.

The most flexible interconnect is a bus-based structure. A *bus* is a set of common copper wires that connect all the units. It supports one writer, and multiple readers at any point of time. For example, unit A can write to the bus at a certain point of time, and all the other units can get the value that unit A writes. It is possible to send data from one unit to another unit, or from one unit to a set of other units if required. The control unit needs to ensure that at any point of time, only one unit writes to the bus, and the unit that needs to process the value that is being written, reads the value from the bus.

Definition 62 *A* bus *is a set of common wires that connect all the functional units in a processor. It supports one writer, and multiple readers at any point of time.*

Let us now proceed to design simplified versions of all the units that we introduced for our hardwired processor. These simplified versions can aptly be used in the data path of our microprogrammed processor.

9.5.1 Fetch Unit

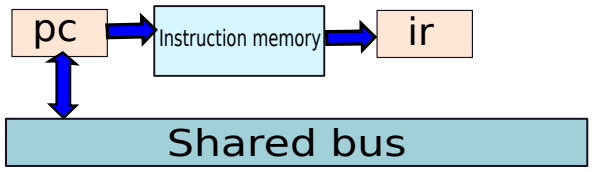


Figure 9.17: The fetch unit in a microprogrammed processor

Let us start out by explaining the design philosophy of the microprogrammed processor. We add registers with every unit. These registers store the input data for the specific unit, and a dedicated output register stores the results generated by the unit. Both these sets of registers are connected to the common bus. Unlike the hardwired processor, where there was a good amount of coupling across different units, units in a microprogrammed processor are fairly independent of each other. Their/ job is to perform a set of actions, and put the results back on to the bus. Each unit is like a function in a programming language. It has an interface consisting of a set of registers to read data in. It typically takes 1 cycle to compute its output, and then the unit writes the output value to an output register.

In concordance with this philosophy, we present the design of the fetch unit in Figure 9.17. It has two registers – *pc* (PC), and *ir* (instruction register). We shall use the acronym, IR, for the instruction register. *ir* contains the contents of the instruction. The *pc* register can read its value from the bus, and can also write its value to the bus. We have not connected *ir* to the bus because no other unit is typically interested in the exact contents of the instruction. Other units are only interested in different fields of the instruction. Hence, it is necessary to decode the instruction and break it into a set of different fields. This is done by the decode unit.

9.5.2 Decode Unit

This unit is similar in function to the operand fetch unit as described in Section 9.2.3. However, we do not include the register file in this unit. We treat it as a separate unit in the microprogrammed processor. Figure 9.18 shows the design of the operand fetch unit.

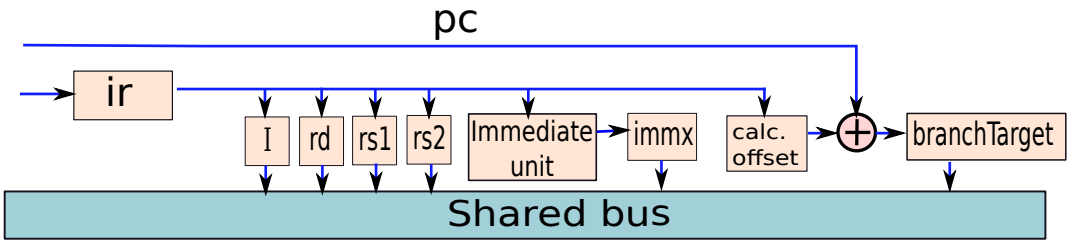


Figure 9.18: The decode unit in a microprogrammed processor

The job of the decode unit is to break down the contents of the instruction into multiple

fields, and export them as registers. In specific, the following registers are made available to the bus, I (immediate bit), rd (destination register), $rs1$ (source register 1), $rs2$ (source register 2), $immx$ (after processing the modifiers), and $branchTarget$ (branch target). To compute the branch target we calculate the offset from the current PC by extracting bits [1:27] from ir, and shifting it to the left by 2 places. This is added to the current value of the PC. Table 9.6 shows the range of bits extracted from ir for each output register.

Register	Bits in ir
I	27
rd	23-26
$rs1$	19-22
$rs2$	15-18
$immx$	1-18 (process modifiers)
$branchTarget$	PC + $(ir[1:27] \ll 2)$

Table 9.6: List of bits in ir corresponding to each register in the decode unit

It is possible that a given program execution might not have values for all the registers. For example, an instruction in the *register* format will not have an embedded immediate value. Hence, in this case the $immx$ register will have junk data. However, it does not hurt to extract all possible fields from the instruction, and store them in registers. We can use only those registers that contain valid data, and ignore those registers that are not relevant to the instruction. This ensures that our data path remains simple, and we do not need costly multiplexers in the decode unit.

9.5.3 Register File

We had combined the decode unit and the register file, into one unit called the operand fetch unit of the hardwired processor. However, we prefer to keep the register file separate in the microprogrammed processor. This is because in the hardwired processor it was accessed right after decoding the instruction. However, this might not be the case in the microprogrammed processor. It might need to be accessed possibly several times during the execution of an instruction.

The register file has two source registers – $regSrc$, and $regData$. The $regSrc$ register contains the number of the register that needs to be accessed. In the case of a write operation, the $regData$ register contains the value to be written. The $args$ values are directly read from the bus. They contain the commands to the register file. We assume that there are dedicated wires in the shared bus to carry the arguments ($args$) values. They take different values, where each value corresponds to a unique operation of a unit. The value 00...0 is a distinguished value that corresponds to a *nop* (no operation).

The arguments to the register file, are very simple – *read*, *write*, and *nop*. If the args specify a *write* operation, then the value in $regData$ is written to the register specified by the $regSrc$ register. If a *read* operation is specified, then the register specified by $regSrc$ is read and its value is stored in the register, $regVal$ (register value).

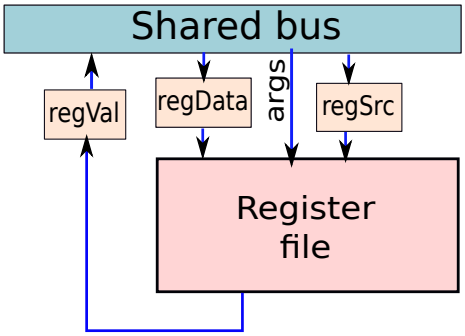


Figure 9.19: The register file in a microprogrammed processor

To access the register file it is thus necessary to write the number of the register to the $regSrc$ register, write the value to be written to the $regData$ register if required, and finally specify the appropriate arguments. The assumption is that after 1 cycle the operation is complete. In the case of a read operation, the value is available in the $regVal$ register.

9.5.4 ALU

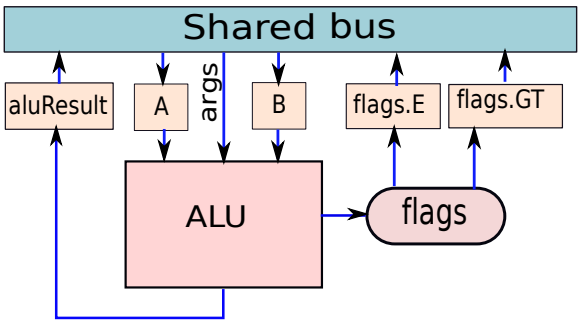


Figure 9.20: The ALU in a microprogrammed processor

The structure of the ALU is shown in Figure 9.20. It has two input registers, A and B. The ALU performs actions on the values contained in registers, A and B. The nature of the operation is specified by the $args$ value. For example, if it specifies an add operation, then the ALU adds the values contained in registers, A and B. If it specifies a subtract operation, then the value in B is subtracted from the value contained in A. For the *cmp* instruction, the ALU updates the flags. Recall that in *SimpleRisc* we use two flags that specify the equality, and greater than conditions. They are saved in the registers $flags.E$ and $flags.GT$, respectively. The result of the ALU operation is then saved in the register $aluResult$. Here also, we assume that the ALU takes 1 cycle to execute after the args values are specified on the bus.

9.5.5 Memory Unit

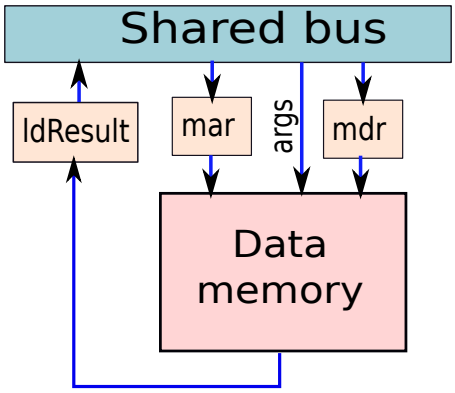


Figure 9.21: The memory unit in a microprogrammed processor

The memory unit is shown in Figure 9.21. Like the hardwired processor, it has two source registers – *mar* and *mdr*. The memory address register(*mar*) buffers the memory address, and the memory data register (*mdr*) buffers the value that needs to be stored. Here also, we require a set of arguments that specify the nature of the memory operation – load or store. Once, a load operation is done, the data is available in the *ldResult* register.

9.5.6 Overview of the Data Path

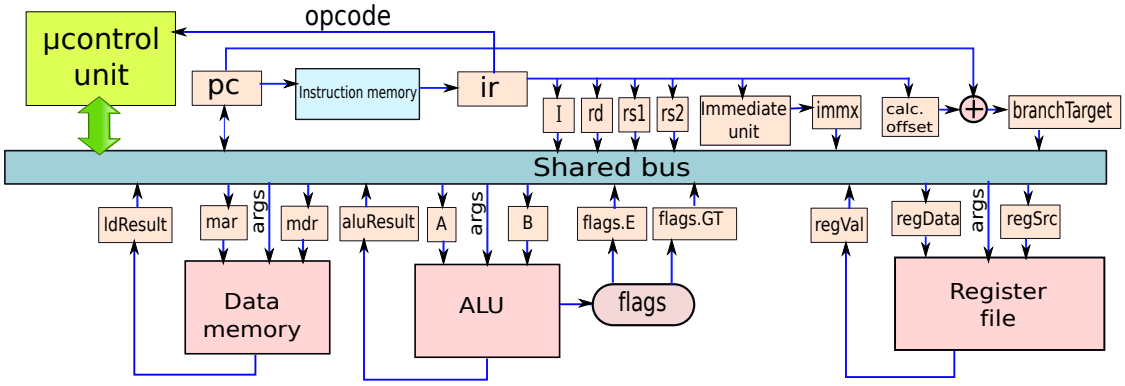


Figure 9.22: The data path in a microprogrammed processor

Let us now add all the individual units, and take a look at the entire data path as shown in Figure 9.22. Along with all the units that we just described, we have added an extra unit, which is the microprogrammed control unit (μcontrol unit). Its role is to execute a set of microinstructions corresponding to each program instruction, and orchestrate the flow of data

values across the different units in the data path of the microprogrammed processor. It is mainly responsible for the execution of microinstructions, data transfers across the different units, and for transferring control to the correct program instruction by updating the PC. Note that we have also added an extra connection between the *ir* register and the μcontrol unit to transfer the opcode of the instruction. We require the μcontrol unit to load the appropriate set of microinstructions corresponding to the program instruction. By design, we do not wish to make the value of the opcode available to other units. This is because, we have a set of microinstructions for each opcode, and there is no reason why other units should require the value of the opcode.

Definition 63

The microcontrol unit*, also referred to as the μcontrol unit is a dedicated piece of hardware that is responsible for the execution of a set of microinstructions corresponding to each program instruction. Its role is to fetch the appropriate set of microinstructions from a dedicated* microprogram memory*, and execute them in sequence. A register called the* micro PC *(μpc) points to the currently executing microinstruction.*

We envision a microprogram memory that is a part of the μcontrol unit. It contains the set of microinstructions corresponding to each program instruction. It is thus necessary for the μcontrol unit to jump to the starting address of the set of microinstructions corresponding to each program instruction. We also need a microPC that points to the current microinstruction being executed.

Before discussing the design and implementation of the μcontrol unit, let us first look at programming, or rather microprogramming our new processor. We need to design a microassembly language that will help us write programs for it.

9.6 Microassembly Language

9.6.1 Machine Model

All the internal registers in Figure 9.22 are the set of registers that are visible to microassembly instructions. Ideally microassembly instructions are not supposed to be aware of regular architectural registers, and other aspects of architectural state. They are only supposed to be aware of internal registers that are not externally visible.

Table 9.7 shows the list of internal registers in our microprogrammed data path. Note that we have 1-bit registers, 4-bit registers, and 32-bit registers.

Microprogrammed instructions do not access memory. Hence, they do not need a view of memory.

9.6.2 Microinstructions

Let us look at the life cycle of a regular program instruction. The first step is to fetch the contents of the instruction from the instruction memory. Let us introduce a microinstruction

Serial No.	Register	Size (bits)	Function
1	pc	32	program counter
2	ir	32	instruction register
3	I	1	immediate bit in the instruction
4	rd	4	destination register id
5	$rs1$	4	id of the first source register
6	$rs2$	4	id of the second source register
7	$immx$	32	immediate embedded in the instruction (after processing modifiers)
8	$branchTarget$	32	branch target, computed as the sum of the PC and the offset embedded in the instruction
9	$regSrc$	4	contains the id of the register that needs to be accessed in the register file
10	$regData$	32	contains the data to be written into the register file
11	$regVal$	32	value read from the register file
12	A	32	first operand of the ALU
13	B	32	second operand of the ALU
14	$flags.E$	1	the equality flag
15	$flags.GT$	1	the greater than flag
16	$aluResult$	32	the ALU result
17	mar	32	memory address register
18	mdr	32	memory data register
19	$ldResult$	32	the value loaded from memory

Table 9.7: List of all the internal registers

to read the contents of the instruction from the instruction memory and place it in the IR (ir). Let us call it $mloadIR$. Note that we will add the prefix m (m for micro) to every microinstruction. This is to denote the fact that it is a microinstruction, and differentiate it from regular program instructions.

Microinstruction	Semantics
$mloadIR$	Loads the ir with the contents of the instruction

Once, we have loaded the instruction register, it automatically sends the contents to all the subunits in the decode unit, and they extract the appropriate bit fields, and save them in the decode registers – I, rd, $rs1$, $rs2$, $immx$, and $branchTarget$. We envision an *mdecode* instruction in the 0-address format that makes the μcontrol unit wait for 1 cycle. In this cycle, all the decode registers get populated.

Microinstruction	Semantics
mdecode	Waits for 1 cycle. Meanwhile, all the decode registers get populated.

Note that these two steps (*mloadIR* and *mdecode*) are common for all program instructions. After this, we need to load the microinstructions for the specific program instruction. This is achieved through an *mswitch* instruction that instructs the μcontrol unit to jump to the appropriate location in the microinstruction memory, and begins executing microinstructions starting from that location.

Microinstruction	Semantics
mswitch	Load the set of microinstructions corresponding to the program instruction

Now, the processing of the instruction can start. The aim here is to use as few microinstructions as possible. We want to keep the microassembly interface very simple. Let us first introduce the *mmov* instruction that moves data from the source register to a destination register. Additionally, it can set the arguments of the unit corresponding to the destination register. We thus introduce a 2-address and 3-address format of the *mmov* instruction. The 3-address format contains the arguments ($args$) of the unit corresponding to the destination register, as shown below.

Microinstruction	Semantics
mmov $r1$, $r2$	$r1 \leftarrow r2$
mmov $r1$, $r2$, $\langle args \rangle$	$r1 \leftarrow r2$, send the value of $args$ on the bus

We sometimes face the need to load constants into registers. Hence, we introduce the *mmovi* instruction that loads a constant into a register.

Microinstruction	Semantics
mmovi $r1$, $\langle imm \rangle$	$r1 \leftarrow imm$
mmovi $r1$, $\langle imm \rangle$, $\langle args \rangle$	$r1 \leftarrow imm$, send the value of $args$ on the bus

We need an *madd* instruction because we need to increment the values of registers such as the *pc*. Instead of using the main ALU, we can have a small adder as a part of the μcontrol unit. We refer to this as the μadder. Here, there is a tradeoff to make. Do we need an add instruction that adds two registers, and saves it in another register? At the microinstruction

level, this is seldom required. We definitely do not require this instruction to implement the *SimpleRisc* instruction set. Hence, we do not see a reason to include this microinstruction. If there is ever a need to have one such microinstruction, then we can always use the main ALU in the data path to perform the addition. We thus introduce a simple add instruction in the 2-address format. It adds an immediate value to a register. The semantics of this instruction is shown below.

Microinstruction	Semantics
madd $r1$, $\langle imm \rangle$	$r1 \leftarrow r1 + imm$
madd $r1$, $\langle imm \rangle$, $\langle args \rangle$	$r1 \leftarrow r1 + imm$, send the value of $\langle args \rangle$ on the bus

Here, the *madd* instruction adds *imm* to $r1$, and saves the result in $r1$. *imm* can be a positive or a negative number. We restrict it to a 12-bit number, because we do not need more bits in most cases. The range of the immediate is thus between -2048 and +2047.

Lastly, we need branch instructions. We need both conditional branches, and unconditional branches. We thus introduce two new microinstructions – *mb* (branch) and *mbeq* (branch if the arguments are equal). The *mb* microinstruction takes a single argument, which is the address of the target microinstruction (or its label while writing microassembly code). We use the PC-direct addressing mode here as compared to the PC-relative addressing mode because, we expect the total number of microinstructions to be small. Secondly, if we would have use a PC-relative addressing mode, then we would have required an extra adder in our data path to add the offset to the PC. The *SimpleRisc* instruction set allocates 5 bits for the opcode. This means that at the most we can have 32 instructions in our instruction set. Let us assume that in the worst case, an instruction translates to 20 microinstructions. We would thus need to store 640 microinstructions. We can thus allocate 10 bits for the specifying the address of the microinstruction and our μpc (micro-PC) can also be 10 bits wide. This means that at the most we can support a total of 1024 microinstructions. This is much more than what we actually require. However, it is not a bad idea to over design hardware because it cannot be changed later. Note that in the microinstruction memory, the address refers to the index of the microinstruction (not to the starting address of the first byte).

The *mbeq* instruction requires three operands. The first operand is a register, the second operand is an immediate, and the third operand is the address(label) of a microinstruction. If the value contained in the register is equal to the immediate operand, then the microPC jumps to the microinstruction specified by the third operand. Otherwise, the next microinstruction in sequence is executed.

Microinstruction	Semantics
mb $\langle addr \rangle$	execute the microinstruction at $\langle addr \rangle$(label) in the microprogram memory
mbeq *reg*, *imm*, $\langle addr \rangle$	If the value in the internal register *reg* is equal to *imm*, then the microPC needs to jump to $\langle add \rangle$(label)

Serial No.	Microinstruction	Semantics
1	$mloadIR$	$ir \leftarrow [pc]$
2	$mdecode$	populate all the decode registers
3	$mswitch$	jump to the μpc corresponding to the opcode
4	$mmov\ reg1, reg2, \langle args \rangle$	$reg1 \leftarrow reg2$, send the value of $args$ to the unit that owns $reg1$, $\langle args \rangle$ is optional
5	$mmovi\ reg1, imm, \langle args \rangle$	$reg1 \leftarrow imm$, $\langle args \rangle$ is optional
6	$madd\ reg1, imm, \langle args \rangle$	$reg1 \leftarrow reg1 + imm$, $\langle args \rangle$ is optional
7	$mbeq\ reg1, imm, \langle addr \rangle$	if $(reg1 = imm)$ $\mu pc \leftarrow addr(label)$
8	$mb\ \langle addr \rangle$	$\mu pc \leftarrow addr(label)$

Table 9.8: List of microinstructions

To summarize, Table 9.8 shows the 8 microinstructions that we have described in this section. We have a compact list of 8 microinstructions, and thus we can encode each microinstruction using just 3 bits.

9.6.3 Implementing Instructions in the Microassembly Language

Let us now try to implement program instructions in the microassembly language using the set of basic microinstructions enumerated in Table 9.8.

For all the instructions, they start with a common set of microinstructions as shown below. We refer to these 4 microinstructions as the *preamble*.

```
1 .begin:
2 mloadIR
3 mdecode
4 madd pc, 4
5 mswitch
```

Definition 64
A set of microinstructions that is common to all program instructions and is executed at the beginning before proceeding to implement the logic of the instruction, is known as the preamble*, or* microcode preamble*.*

Every instruction needs to pass through at least three of these steps. We need to fetch the contents of the PC and load them into the *ir* register. Then, we need to decode the instruction, and break it down into its constituent fields. For instructions, other than branches, we need to increment the value of the PC by 4. In our microcode we prefer to do this step for all the instructions. For taken branches, we need to later overwrite the PC with the branch target. Lastly, we need to execute the *mswitch* instruction to jump to the starting location of the set of microinstructions that are specific to the program instruction.

The label *.begin* points to the beginning of this routine. Note that after finishing the execution of an instruction, we need to jump to the *.begin* label such that we can start processing the next instruction in the program. Note that in our microassembly code we specify the label that we need to branch to. When the microassembly code is translated to actual machine level microinstructions, then each label is replaced by the address of the corresponding microinstruction.

9.6.4 3-Address Format ALU Instructions

Let us now look at implementing 3-address format ALU instructions. These instructions are: *add*, *sub*, *mul*, *div*, *mod*, *and*, *or*, *lsl*, *lsr*, and *asr*.

First, we need to read the value of the first operand stored in $rs1$ from the register file, and send it to the ALU. The microcode snippet to achieve this is as follows:

```
1 mmov regSrc, rs1, <read>
2 mmov A, regVal
```

Note, that we are combining a functional unit operation, and a register transfer in the same cycle. This can be confusing at the beginning. Hence, the reader should read this example several times and ensure that she has a clear understanding. The reason that we fuse both the operations is because microcode registers are typically very small, and thus they can be accessed very quickly. Hence, it is not a good idea to use a complete cycle for transferring data between micro registers. It is a better idea to fuse a register transfer with a functional unit operation, such that we can ensure that we are roughly doing a similar amount of work every cycle.

Let us proceed. Subsequently, we need to check if the second operand is a register or an immediate. This can be achieved by comparing the I register with 1. If it is 1, then the second operand is an immediate, else it is a register. The following piece of code first checks this condition, and then performs data transfers accordingly.

```
1 mbeq I, 1, .imm
2 /* second operand is a register */
3 mmov regSrc, rs2, <read>
4 mmov B, regVal, <aluop>
5 mb .rw
6 /* second operand is an immediate */
7 .imm:
8 mmov B, immx, <aluop>
```

```
/* write the ALU result to the register file*/
.rw:
```

Here, we first check if the value stored in the I register is equal to 1, using the *mbeq* instruction. If it is not 1, then the second operand is a register, and we start executing the subsequent microinstruction. We move the contents of the register, $rs2$, to the $regSrc$ register that contains the index of the register that we need to read from the register file. Then we move the value of the operand read from the register file ($regVal$) to the ALU (register B). Since the value in register A is already present, we can directly start the ALU operation. This is indicated to the ALU by sending an extra argument ($\langle aluop \rangle$) that encodes the ALU operation. $\langle aluop \rangle$ corresponds to one of the following operations: *add*, *sub*, *mul*, *div*, *mod*, *and*, *or*, *lsl*, *lsr*, and *asr*.

However, if the value of the I register is 1, then we need to branch to *.imm*. The value of the immediate embedded in the instruction is already available with appropriate sign extensions in the register $immx$. We need to simply transfer the value of $immx$ to B (second ALU register), and the arguments ($\langle aluop \rangle$) to the ALU. Similar to the case with the second operand being a register, $\langle aluop \rangle$ encodes the ALU operation. Once, we are done, we need to start execution at the label, *.rw*.

The label *.rw* needs to point to code that writes the value of the computed result to the register file, and then proceeds to execute the next instruction. The code for these two operations is shown below.

```
.rw:
        mmov regSrc, rd
        mmov regData, aluResult, <write>
        mb .begin
```

We write the result of the ALU into the register file, and then branch to the beginning, where we proceed to execute the next instruction. To summarize, here is the code for any 3-address format ALU instruction (other than the preamble).

```
/* transfer the first operand to the ALU */
mmov regSrc, rs1, <read>
mmov A, regVal

/* check the value of the immediate register */
mbeq I, 1, .imm
/* second operand is a register */
mmov regSrc, rs2, <read>
mmov B, regVal, <aluop>
mb .rw
/* second operand is an immediate */
.imm:
mmov B, immx, <aluop>

```

```
15 /* write the ALU result to the register file*/
16 .rw:
17 mmov regSrc, rd
18 mmov regData, aluResult, <write>
19 mb .begin
```

This code snippet has 10 microinstructions. Recall that we also need to execute 4 more microinstructions as a part of the preamble before this. They read the PC, decode the instruction, set the next PC, and jump to the beginning of the appropriate set of microinstructions. Executing 14 microinstructions for 1 program instruction is clearly a lot of effort. However, the reader must recall that we are not really after performance here. We wanted to design a very clean and flexible means of accessing different units.

9.6.5 2-Address Format ALU Instructions

The three ALU instructions in the 2-address format, are *not*, *mov*, and *cmp*. *not* and *mov* have a similar format. They do not use the first source operand, $rs1$. They operate on either $rs2$, or $immx$, and transfer the result to the register pointed by rd.

Let us look at the *mov* instruction first. We first check whether the second operand is an immediate, or not, by comparing the value in register I with 1. If it is equal to 1, then we jump to the label, *.imm*. Otherwise, we proceed to execute the subsequent instructions in Lines 4, and 5. In Line 4, we transfer $rs2$ to $regSrc$, along with the $\langle read \rangle$ command. The operand is read and stored in $regVal$. In the next cycle, we transfer $regVal$ to $regData$ such that it can be written back to the register file. If the second operand was an immediate, then we execute the code in Line 9. We transfer the immediate (stored in $immx$) to the $regData$ register. In either case, $regData$ contains the value to be written to the register file. Then we transfer the id of the destination register (stored in rd) to $regSrc$, and simultaneously issue the write command in Line 13.

mov instruction

```
1  /* check the value of the immediate register */
2  mbeq I, 1, .imm
3  /* second operand is a register */
4  mmov regSrc, rs2, <read>
5  mmov regData, regVal
6  mb .rw
7  /* second operand is an immediate */
8  .imm:
9  mmov regData, immx
10
11 /* write to the register file*/
12 .rw:
13 mmov regSrc, rd, <write>
14
15 /* jump to the beginning */
16 mb .begin
```

Let us now write a similar routing for the *not* instruction. The only additional step is to transfer the value read from the register to the ALU, compute the logical negation, and then transfer the value back to the register file. A hallmark feature of our microassembly language is that we can transfer a value to a unit, and if all the other operands are in place, then we can also perform an operation in the unit in the same cycle. The implicit assumption here is that 1 clock cycle is enough to transfer the data between registers, and perform a computation. In line with this philosophy we transfer the value of $immx$, or $regVal$ to register B of the ALU, and also perform a $\langle not \rangle$ operation in the same cycle (see Lines 5 and 9). Like the *mov* instruction, we transfer the ALU result to the $regData$ register, and write it to the register file in Lines 13, and 14.

not instruction

```
1  /* check the value of the immediate register */
2  mbeq I, 1, .imm
3  /* second operand is a register */
4  mmov regSrc, rs2, <read>
5  mmov B, regVal, <not> /* ALU operation */
6  mb .rw
7  /* second operand is an immediate */
8  .imm:
9  mmov B, immx, <not> /* ALU operation */
10
11 /* write to the register file*/
12 .rw:
13 mmov regData, aluResult
14 mmov regSrc, rd, <write>
15
16 /* jump to the beginning */
17 mb .begin
```

Let us now look at the compare instruction that does not have a destination operand. It compares two operands, where one is a register operand, and the other can be either a register or an immediate. It saves the results automatically in the $flags.E$ and $flags.GT$ registers.

Let us now consider the microcode for the *cmp* instruction.

cmp instruction

```
1  /* transfer rs1 to register A */
2  mmov regSrc, rs1, <read>
3  mmov A, regVal
4
5  /* check the value of the immediate register */
6  mbeq I, 1, .imm
7  /* second operand is a register */
8  mmov regSrc, rs2, <read>
9  mmov B, regVal, <cmp> /* ALU operation */
10 mb .begin
11
```

```
12 /* second operand is an immediate */
13 .imm:
14 mmov B, immx, <cmp> /* ALU operation */
15 mb .begin
```

Here, we first transfer the value in register $rs1$ to the ALU (in register A). Then, we check if the second operand is an immediate. If it is an immediate, then we transfer the value of $immx$ to the ALU (in register B), and simultaneously issue a command to execute a compare in Line 14. However, if the second operand is a register, then we need to read it from the register file (Line 8), and then transfer it to the ALU (Line 9). The last step is to branch to the beginning (*mb .begin*).

9.6.6 The *nop* Instruction

Implementing the *nop* instruction is trivial. We just need to branch to the beginning as shown below.

```
1 mb .begin
```

9.6.7 *ld* and *st* instructions

Let us now look at the load instruction. We need to transfer the value of the first source register to the ALU. Then we transfer the value of the immediate to the second ALU register (B), and initiate the add operation to calculate the effective address. Once the effective address has been calculated, it is available in the *aluResult* register. Subsequently, we move the contents of the *aluResult* register to the memory address register (*mar*), and initiate a load operation. The result is available in the *ldResult* register in the next cycle. We write the loaded value to the register specified by rd in the next two cycles.

ld instruction

```
1  /* transfer rs1 to register A */
2  mmov regSrc, rs1, <read>
3  mmov A, regVal
4
5  /* calculate the effective address */
6  mmov B, immx, <add> /* ALU operation */
7
8  /* perform the load */
9  mmov mar, aluResult, <load>
10
11 /* write the loaded value to the register file */
12 mmov regData, ldResult
13 mmov regSrc, rd, <write>
14
15 /* jump to the beginning */
16 mb .begin
```

The microcode for the store instruction is similar to that of the load instruction. We first calculate the effective memory address and store it in the *mar* register. Then we read the value of the *rd* register that contains the data to be stored (Line 10). We save this in the *mdr* register, and issue the store (Line 11).

st instruction

```
1  /* transfer rs1 to register A */
2  mmov regSrc, rs1, <read>
3  mmov A, regVal
4
5  /* calculate the effective address */
6  mmov B, immx, <add> /* ALU operation */
7
8  /* perform the store */
9  mmov mar, aluResult
10 mmov regSrc, rd, <read>
11 mmov mdr, regVal, <store>
12
13 /* jump to the beginning */
14 mb .begin
```

9.6.8 Branch Instructions

There are five branch instructions in *SimpleRisc* : *b*, *beq*, *bgt*, *call*, and *ret*.

Implementing the unconditional branch instruction is trivial. We simply need to transfer the value of the branch target to the PC.

b instruction

```
1  mmov pc, branchTarget
2  mb .begin
```

We can make a minor modification to this code to implement the *beq*, and *bgt* instructions. We need to check the value of the flags registers, and set the branchTarget to the PC only if the corresponding flags register contains a 1.

beq instruction

```
1  /* test the flags register */
2  mbeq  flags.E, 1, .branch
3  mb. begin
4
5  .branch:
6  mmov pc, branchTarget
7  mb .begin
```

bgt instruction

```
1  /* test the flags register */
2  mbeq  flags.GT, 1, .branch
3  mb. begin
4
5  .branch:
6  mmov pc, branchTarget
7  mb .begin
```

The last two instructions that we need to implement are the *call* and *ret* instructions. The *call* instruction is a combination of a simple branch, and a register write operation that adds the value of the next PC ($PC + 4$) to the return address register (register 15). The microcode

is as follows. Note that we do not increment the PC by 4 because it is already incremented in the preamble.

call instruction

```
1 /* save PC + 4 in the return address register */
2 mmov regData, pc
3 mmovi regSrc, 15, <write>
4
5 /* branch to the function */
6 mmov pc, branchTarget
7 mb .begin
```

We save the address of the next PC in the register file in lines 2 to 3. Then we move the *branchTarget* to the PC, and then proceed to execute the first instruction in the invoked function.

The *ret* instruction performs the reverse operation, and transfers the return address to the PC.

ret instruction

```
1
2 /* save the contents of the return
3                 address register in the PC */
4 mmovi regSrc, 15, <read>
5 mmov pc, regVal
6 mb .begin
```

We have thus implemented all our *SimpleRisc* instructions in microcode. A microcoded implementation is definitely slower that our hardwired data path. However, we have gained a lot in terms of flexibility. We can implement some very complex instructions in hardware, and thus make the task of software developers significantly easier. We can also dynamically change the behavior of instructions. For example, if we wish to store the return address on the stack rather than the return address register, we can do so easily (see Examples 132 and 133).

Example 132

Change the call instruction to store the return address on the stack. The preamble need not be shown (study carefully).

Answer:

stack-based call instruction

```
1
2 /* read and update the stack pointer */
3 mmovi regSrc, 14, <read> /* regSrc contains the id
4                   of the stack pointer */
5 madd regVal, -4 /* decrement the stack pointer */
6 mmov mar, regVal /* MAR contains the new stack pointer */
7
```

```
mmov regData, regVal, <write> /* update the stack pointer */

/* write the return address to the stack */
mmov mdr, pc, <store>

/* branch to the new function */
mmov pc, branchTarget
mb. begin
```

Example 133

Change the ret instruction to load the return address from the stack. The preamble need not be shown.

Answer:

stack-based call instruction

```
/* read the stack pointer */
mmovi regSrc, 14, <read>

/* set the memory address to the stack pointer */
mmov mar, regVal, <load>

mmov pc, ldResult /* set the PC */

/* update the stack pointer */
madd regVal, 4 /* sp = sp + 4 */
mmov regData, regVal, <write> /* update stack pointer */

/* jump to the beginning */
mb .begin
```

Example 134

Implement an instruction to compute the factorial of the number saved in register $r2$. You can destroy the contents of $r2$. Save the result in register $r3$. Assume that the number is greater than 1.

stack-based call instruction

```

/* code to set the inputs to the multiplier */
mmovi B, 1
mmovi regSrc, 2, <read>
```

```
mmov  A, regVal
/* at this point A = r2, B = 1 */

/* loop */
.loop:
/* Now begin the multiplication */
mmov B, B, <multiply> /* aluResult = A * B */
mmov B, aluResult /* B = aluResult */

/* decrement and test */
madd A, -1 /* A = A - 1 */
mbeq A, 1, .out /* compare A with 1 */
mb .loop

.out:
mmov regData, aluResult
mmovi regSrc, 3, <write> /* all done */

mb .begin
```

Example 135

Implement an instruction to find if the value saved in register $r2$ is a cubic Armstrong Number. A cubic Armstrong number is equal to the sum of the cubes of its decimal digits. For example, 153 is one such number. $153 = 1^3 + 5^3 + 3^3$. Save the Boolean result in $r3$. Assume two scratch registers: $sr1$ and $sr2$.

stack-based call instruction

```

/* Set the inputs of the ALU */
mmovi regSrc, 2, <read>
mmov  A, regVal
mmov  sr1, regVal
mmovi B, 10
mmovi sr2, 0 /* sum = 0 */

/* loop */
.loop:
/* test */
mbeq A, 0, .out

/* compute the mod and cube it */
mmov B, B, <mod> /* aluResult = A % B */
```

```
mmov B, aluResult /* B = aluResult */
mmov A, aluResult, <multiply> /* aluResult = (A%B)^2 */
mmov A, aluResult, <multiply> /* aluResult = (A%B)^3 */
mmov A, aluResult /* A =  (A%B)^3 */
mmov B, sr2, <add> /* add the running sum */
mmov sr2, aluResult /* sr2 has the new sum */

/* test */
mmov A, sr1 /* old value of A */
mmovi B, 10, <divide>
mmov A, aluResult /* A = A / 10 */
mmov sr1, A /* sr1 = A */

mb .loop

/* out of the loop */
.out:
mmov A, sr2 /* A contains the sum */
mmov B, regVal, <cmp> /* compare */
mmov regSrc, 3
mbeq flags.E, 1, .success

/* failure */
mmov regData, 0, <write>
mb .begin

.success:
mmov regData, 1, <write>
mb .begin
```

Example 136

Implement an instruction to test if a number saved in register r2 is prime. Assume that the number is greater than 3. Save the result in r3.

stack-based call instruction

```

/* Read the register and set the ALU inputs */
mmovi regSrc, 2, <read>
mmov  A, regVal
mmovi B, 1

.loop:
```

```
8         /* test for divisibility */
9         madd B, 1, <mod> /* aluResult = A % (B+1), B = B + 1 */
10        mbeq aluResult, 0, .failure
11
12        /* test B */
13        mmov A, A, <cmp> /* compare A with B */
14        mbeq flags.E, 1, .success
15
16 mb .loop
17
18 .success:
19        mmovi regSrc, 3
20        mmovi regData, 1, <write>
21        mb .begin
22
23 .failure:
24        mmovi regSrc, 3
25        mmovi regData, 0, <write>
26        mb. begin
```

9.7 Shared Bus and Control Signals

Let us take a look again at the list of implemented microinstructions in Table 9.8. We observe that each microinstruction has at the most one register read operand, and one register write operand. We typically read from one internal register, and then use it as a part of a computation (addition or comparison), and then write the result to another internal register.

We thus propose the design of a shared bus that actually consists of two buses as shown in Figure 9.23. The first bus is known as the write bus that is connected to all the registers that might potentially write data to the bus. The output of the write bus, the embedded immediate (μimm) in the microinstruction, and the output of the μadder are sent to a multiplexer. Recall that the μadder adds the embedded immediate with the contents of a register. Now, this multiplexer chooses one value among the three, and then sends it on the read bus. We refer to this multiplexer as the *transfer multiplexer*. All the registers that might potentially read a value are connected to the read bus. The PC is connected to both the buses. The μadder has two inputs. One of them is the sign extended immediate that is a part of the microinstruction, and the other is the output of the write bus.

Simultaneously, we compare the value sent on the write bus with the embedded immediate (μimm). The result is contained in the $isMBranch$ signal. The $isMBranch$ signal is required for implementing the *mbeq* instruction.

To create a flexible data path, we need to add as many interconnections between units as possible. We thus decide to connect every register other than the decode, and flags registers to both the read and write buses. These registers are the input/output registers of the register

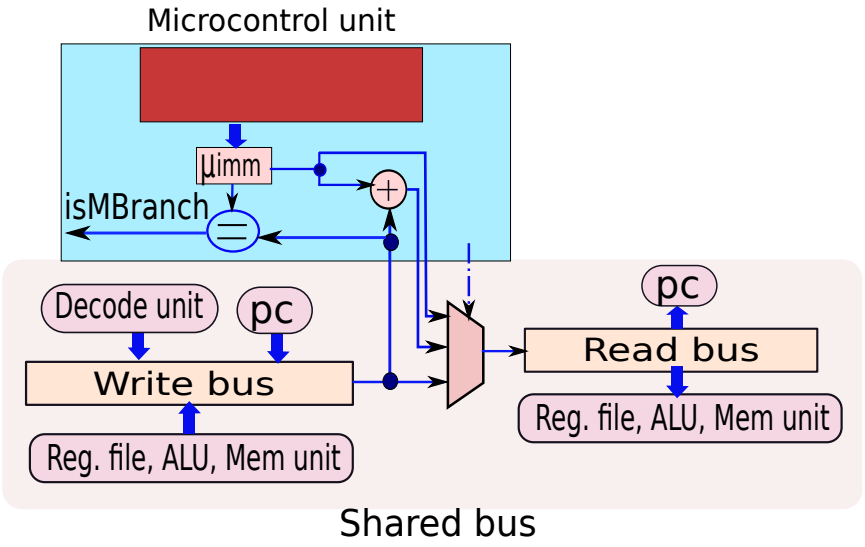


Figure 9.23: The design of the shared bus

file ($regSrc$, $regData$, and $regVal$), the ALU registers (A, B, $aluResult$), and the registers associated with the memory unit (mar, mdr, $ldResult$). To support branch instructions, it is also necessary to connect the PC to both the buses.

9.7.1 Control Signals

Each register that writes to the write bus needs a control signal. If it is asserted (equal to 1), then the value of the register appears on the write bus. Otherwise, the value of the register does not get reflected on the write bus. For example, the register, $aluResult$, contains the result of an ALU operation, and it is sometimes necessary to transfer its value to the write bus. The signal $aluResult_{out}$ controls the behavior of the aluResult register. We associate similar signals with the subscript, $_{out}$ with all the registers that need to access the write bus.

Likewise, we associate a set of signals with the registers that are connected to the read bus. For example, the register mar is connected to the read bus. We associate the signal mar_{in} with it. If it is 1, then the value of the data on the read bus is transferred to mar. If $mar_{in} = 0$, the mar register is effectively disconnected from the read bus.

The PC has two signals associated with it: pc_{in} and pc_{out}. The μcontrol unit ensures that at one point of time only one register can write to the write bus. However, it is theoretically possible for multiple registers to read from the read bus concurrently.

9.7.2 Functional Unit Arguments

We augment the read bus to carry the arguments for the functional units (referred to as $\langle args \rangle$). These arguments specify the nature of the operation, which the functional unit needs to perform. For example, the two operations associated with the memory unit are $\langle load \rangle$, and $\langle store \rangle$, and the two operations associated with the register file are $\langle read \rangle$ and $\langle write \rangle$. Each ALU operation

also has its separate code.

We propose to encode each operation in binary, and reserve the special value of 0 to indicate that no operation needs to be performed. Each functional unit needs to be connected to the read bus, and needs to process the value of the arguments. The $\langle args \rangle$ field can be split into two parts: $\langle unit\ id \rangle$, and $\langle opcode \rangle$. The $\langle unit\ id \rangle$ specifies the identifier for the functional unit. For example, we can assign 00 to the ALU, 01 to the register file, and 10 to the memory unit. The $\langle opcode \rangle$ contains the details of the operation to be performed. This is specific to the functional unit. We propose a 10-bit $\langle args \rangle$ bus that is a part of the read bus. We devote 3 bits to the $\langle unit\ id \rangle$, and 7 bits to the $\langle opcode \rangle$. Thus, for each unit we can support 128 different operations. Implementing the circuit to process the $\langle args \rangle$ is easy, and we leave it as an exercise to the reader.

9.8 The Microcontrol Unit

We now arrive at the last piece of our microprogrammed processor, which is the design of the μcontrol unit. It is a simple processor that executes microinstructions. It consists of a μfetch unit and a μpc. Every cycle we increment the μpc by 1 (addressed by the number of the instruction, not by bytes), or set it to the branch target. Then, we proceed to read the microinstruction from the microprogram memory, and process it. There are two main paradigms for designing an encoding of microinstructions, and executing them using a μcontrol unit. The first is known as *vertical microprogramming*. In principle, this paradigm is similar to executing regular program instructions using a hardwired processor. The second paradigm is known as *horizontal microprogramming*. This is more common, and is also a more efficient.

9.8.1 Vertical Microprogramming

In vertical microprogramming, we encode an instruction similar to encoding a regular RISC instruction in a hardwired processor.

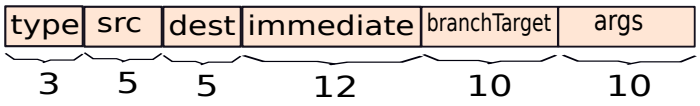


Figure 9.24: Encoding of a microinstruction (vertical microprogramming)

Figure 9.24 shows an encoding for our scheme. Here, we devote 3 bits for encoding the type of the microinstruction. We need 3 bits because we have a total of 8 microinstructions (see Table 9.8). Each microinstruction embeds the 5 bit id (because we have 19 registers visible to microprograms) of an internal source register, 5 bit id of an internal destination register, a 12-bit immediate, and a 10-bit branch target. At the end, we encode a 10-bit args value in the microinstruction. Each instruction thus requires 45 bits.

Now, to process a vertically encoded microinstruction, we need a dedicated μdecode unit that can generate all the control signals. These signals include all the enable signals for the

internal registers, and the signals to select the right input in the transfer multiplexer. Additionally, it needs to extract some fields from the microinstruction such as the immediate, branch target, and the *args* value, and subsequently extend their sign. We have already gone through a similar exercise for extracting the fields of an instruction, and generating control signals, when we discussed the operand fetch unit and control unit for our hardwired processors in Sections 9.2.3, and 9.3, respectively. The logic for generating the control signals, and extracting fields from the microinstruction is exactly the same. Hence, we leave the detailed design of these units as an exercise for the reader.

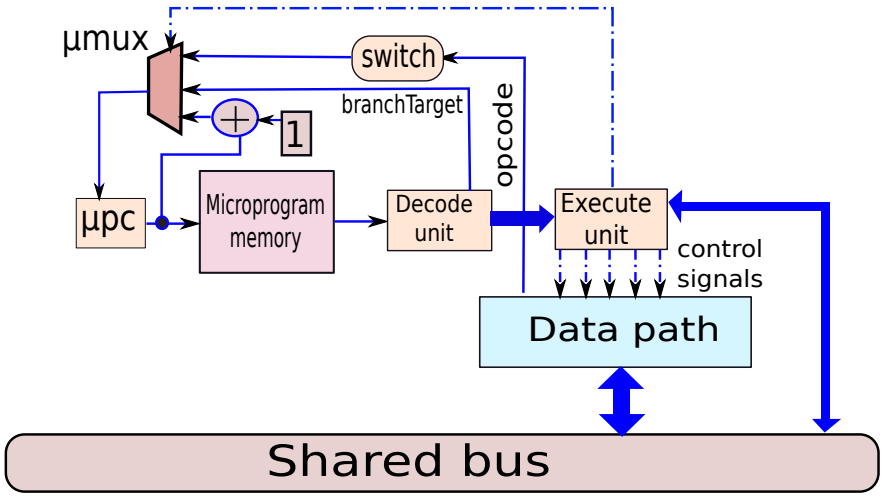


Figure 9.25: The μcontrol unit (vertical microprogramming)

The design of the vertical μcontrol unit is shown in Figure 9.25. We have a microPC (μpc), which saves the index of the currently executing microinstruction. Every cycle, we increment the μpc by 1. This is because each row of the microprogram memory saves 1 microinstruction. We assume that a row is wide enough to save the entire contents of the microinstructions. We do not have the requirement of saving data at the granularity of a fixed number of bytes here. After reading the microinstruction, we proceed to decode it. The process of decoding a microinstruction breaks it into a set of fields (instruction type, immediate, branch target, *args*, source, and destination registers). Subsequently, we generate all the control signals and dispatch the set of control signals to the execute unit. The execute unit sets all the control signals, and orchestrates an operation in the data path of the processor. The execute unit also sets the control signals of the transfer multiplexer. We need some additional support to process the *mswitch* instruction. We add a dedicated switch unit that takes inputs (the opcode) from the *ir* register, and computes the starting address for the microcode of the currently executing program instruction. It sends the address to the multiplexer, $\mu fetch$ (see Figure 9.25). The multiplexer chooses between three inputs – default microPC, branch target, and the address generated by the switch unit. It is controlled by the execute unit. The rules for choosing the input are shown in Table 9.9. In accordance with these rules, and the value of the $isMBranch$ signal (generated by comparing μimm, and the contents of the shared bus), the execute unit

generates the control signals for the $\mu fetch$ multiplexer.

Instruction	Output of the $\mu fetch$ multiplexer
$mloadIR$	next μpc
$mdecode$	next μpc
$mswitch$	output of the switch unit
$mmov$	next μpc
$mmovi$	next μpc
$madd$	next μpc
mb	branch target
$mbeq$	branch target if $isMBranch = 1$, else next μpc

Table 9.9: Rules for controlling the $\mu fetch$ multiplexer

9.8.2 Horizontal Microprogramming

We can further simplify the design of the μcontrol unit. We do not need three steps (fetch, decode, execute) to execute a microinstruction. The decode step is not required. We can embed all the control signals in the microinstruction itself. It is thus not required to have a dedicated signal generator to generate all the control signals. By doing so, we will increase the size of the encoding of an instruction. Since the number of microinstructions is small, and we do not have any significant constraints on the size of the encoding of a microinstruction, adding additional bits in the encoding is not an issue. This paradigm is known as *horizontal microprogramming*. The encoding of a microinstruction is shown in Figure 9.26.

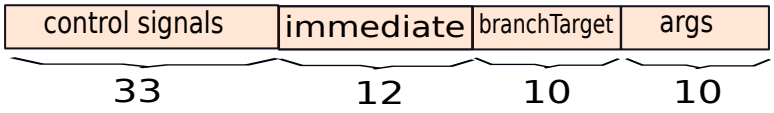


Figure 9.26: Encoding of a microinstruction (horizontal microprogramming)

We need the following fields – control signals (saved as a bit vector whose size is 33 bits), immediate (12 bits), branch target (10 bits), and args (10 bits). The reason we require 33 control signals is as follows. We have 19 registers (see Table 9.7) visible to microcode. Out of these register, the following 9 registers are exclusively connected to either the read bus or the write bus: ir, $flags.E$, $flags.GT$, I, rd, $rs1$, $rs2$, $branchTarget$, and $immx$. Hence, these registers require just one control signal. The rest of the registers have read-write functionality. Hence, these registers require two control signals. Thus, the total number of register enabling control signals are 29. We need 2 more signals each to control the transfer multiplexer, and the $\mu fetch$ multiplexer. We thus have a total of 33 control signals, and we require 65 bits to encode the instruction. Recall that with vertical microprogramming, we needed 45 bits.

Now, with additional storage we can completely eliminate the signal generator in the decode stage, and thus significantly simplify the μcontrol unit as shown in Figure 9.27

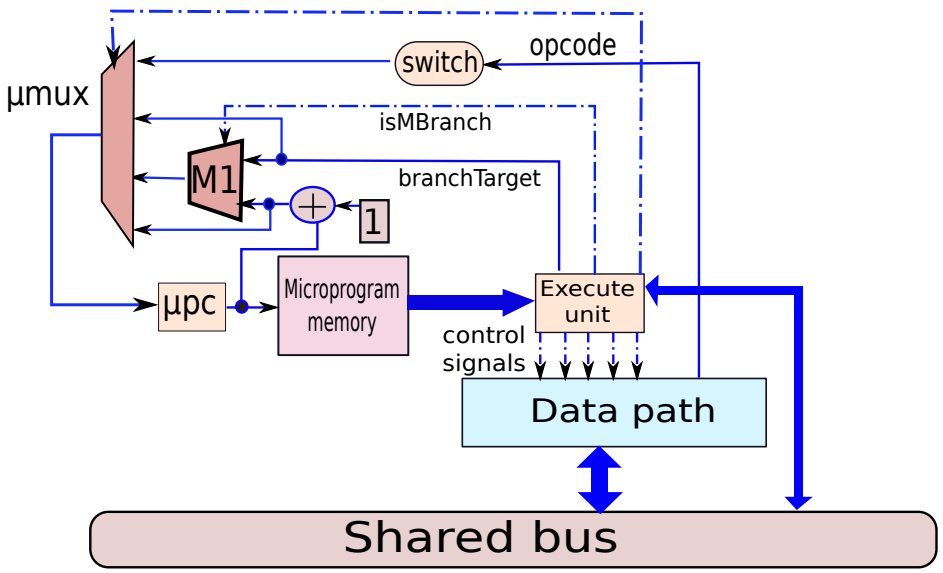


Figure 9.27: The μcontrol unit (horizontal microprogramming)

Here, we have eliminated the decode stage. All the signals are embedded in the instruction, and they are thus used to orchestrate a computation in the data path. The execute unit generates the $isMBranch$ signal (by comparing the μimm and the value on the read bus), which is used to choose between the next μpc, and the branch target using multiplexer, $M1$. Here, we slightly complicate the $\mu fetch$ multiplexer, and add a little bit of redundancy in the interest of simplicity. We make it a 4 input structure, and choose between the value from the switch unit, the branch target, the output of $M1$, and the next μpc. The 2-bit control signals for controlling the $\mu fetch$ multiplexer are embedded in the instruction in accordance with the rules given in Table 9.9. The rest of the operation of the circuit is the same as the circuit for vertical microprogramming as shown in Figure 9.25.

9.8.3 Trade-offs between Horizontal and Vertical Microprogramming

The trade-offs between horizontal and vertical microprogramming are the following:

1. Horizontal microprogramming requires more storage. However, this is not an issue in a microprogrammed processor. The additional storage is minimal.

2. Horizontal microprogramming eliminates the need for dedicated signal generation logic in the μcontrol unit.

3. To program a horizontally microprogrammed processor, it is necessary to expose the control signals to the programmer and the microassembler. This makes the microassembler very specific to a given processor. However, in vertical microprogramming, as long as the internal register set remains the same, we do not need different microassemblers.